

Mohammad Shafiqul Islam

**Dependency of Modal Sense Classification on Different Word Embedding Techniques
using LSTM**



Linguistic Data Science Lab
Ruhr University Bochum
Course/semester: Research Project 1, SuSe 2022
Instructor: Dr. Claudia Roch
Submission date: 13.03.2023

Mohammad.Islam-p2y@ruhr-uni-bochum.de

Abstract

From a very simple point of view, Modal Sense Classification is equivalent to recognizing the meaning of the sentence in terms of the Modal verb's scope. It is quite a difficult task to classify a sentence into a specific modal category, because it is heavily influenced by the context and/or the structure of the sentence. This paper aims to find out how much different word embedding techniques effect classifying modal senses along with LSTM, as different word embedding techniques use different approaches and methodologies to embed a text. It was found that the RoBERTa word embedding technique has a better result compared to the XLNet in terms of the same LSTM model architecture.

1 Introduction

The usage of pre-trained language models has become one of the most popular approaches recently to solving different tasks in the field of Natural Language Processing (NLP). As it is an essential part to vectorize the text for the deep learning model, the motivation of this work was to make use of different vectorizing approaches (Word Embedding Techniques).

To define Modal Sense Classification, it is obvious to look at the definition of modal verbs first. A modal verb is a verb that gives information about the context of a sentence in terms of expression (modal's scope), such as permission, request, order, obligation, etc. Sense refers to the term "meaning" that is carried by a lexical unit in a given context. And finally, Classification means it is an activity to recognize shared features or similar elements and categorize them. So altogether, Modal Sense Classification can be defined as Recognizing the Meaning of a Sentence in terms of the Modal's scope.

Modal Sense Classification can be considered as a Word Sense Disambiguation (WSD) task for the fixed inventory of Modal Verbs. So, the purpose is to disambiguate the sense of a modal verb in a concrete context. However, Word Sense Disambiguation (WSD) is considered AI-Complete, which means it is a task really hard to solve.

To understand why Modal Sense Classification is quite tricky, it is necessary to look into the modality of some examples from the “Modality (linguistics)” (2023) page of [Wikipedia](#).

1. Agatha must be the murderer.
2. Agatha must go to jail.

These two sentences use the same modal verb “must”, but their expressive meaning is not the same. The first sentence shows something like “possibility or a might-be-situation”, and it contains the speaker’s conclusion based on his/her knowledge. On the other hand, the second example shows an “obligatory” situation given the law or norms for the participant in the sentence. So, a modal verb can have different meanings depending on factors like context, readings, and interpretation. These factors are not static and can vary from person to person, which is the main reason that Modal Sense Classification can be hard to achieve.

Depending on the expressive context, there can be three types of Modal Senses.

- Epistemic
- Deontic
- Dynamic

The epistemic category deals with the speaker’s evaluation of the possibility of being true or not true. Considering example sentence (1), it can be said that this sentence has epistemic modality, because it is highly probable that Agatha is the murderer. The deontic modality deals with request and/or permission. The example sentence (2) shows deontic behavior as it is a sort of request to go to jail for Agatha from the speaker’s point of view. Finally, dynamic modality refers to the subject’s own ability to act. To have a better idea of dynamic modality, here is another example:

3. Agatha can sing.

So, the example sentence (3) shows that Agatha herself has the ability to sing.

So, it can be safely said that Modal Sense Classification hugely depends on context. In this paper, I present an experiment that would figure out if using different Word Embedding techniques that use different contextual approaches and contain discriminative information have any effect on Modal Sense Classification with LSTM.

Section 2 discusses some of the related works that have already been done. In section 3, the solution architecture of my approach will be discussed. This section consists of an overview of the environment

setup, dataset, data preparation with different word embedding techniques for the LSTM model, model building, and model evaluation. Then section 4 will present the results and discussion of the experiment. Section 5 will have the conclusion along with the experiment's future scope. Finally, section 6 will have the references that are used for this work.

2 Related Works

It was necessary to look into previous works done by others to gather information regarding what has already been done and what can still be done about Modal Sense Classification. The early research related to Modal Sense Classification was based on automatic classifiers to find the correct modality with some predefined rules or schemas. It seems that not much work has been done in terms of applying Neural Networks. This could be a reason why using different word embedding techniques for the Neural Network model remains out of focus.

Ruppenhofer et al. (2012) presented an annotation scheme for English modal verbs with their modalities from the news domain. They also experimented with automatic modal sense tagging for the texts. They proposed their guidelines for some specific modal verbs: *must*, *should*, *ought to*, *may/might*, and *can*. Apart from the three types of modalities (Epistemic, Deontic, and Dynamic), they also extended it with some other categories, such as Optative, Concessive, and Conditional. Inter-annotator agreement (IAA) between all the annotators was performed to measure different metrics. Finally, the automatic sense tagging confirmed their annotation technique. They experimented with a 10-fold Cross Validation setup with a maximum entropy classifier. This classifier was trained on the annotated data of each modal verb with the training set and predicted the modality of each modal verb in the test set. They found that the accuracy of the classifier was almost the same for each modal verb compared to the percentage of Inter-annotator agreement (IAA). They concluded that their results were quite promising.

Another paper by Zhou et al. (2015) showed how to classify the modal senses by applying features that are related to lexical, proposition-level, and discourse-level semantic factors. These semantic factors played a vital role in the context of the texts. Their main intention was to erase the difficulty of the modal sense distinctions by removing the distributional bias and sparsity of the annotated dataset available. Their dataset was based on the work of Ruppenhofer et al. (2012). But instead of working with all six modalities, they reduced the modalities to three main categories (Epistemic, Deontic, and Dynamic). To do that, they subsumed Conditional and Concessive modalities to Epistemic and Optative to Deontic. For the semantic features, they had three feature groups: Embedded verb, Subject noun Phrase, and Sentence structure. Each of these groups also consisted of individual sub-features. They replicated the classifiers used by Ruppenhofer et al. (2012) for the classification with different training data and feature sets. Finally, they concluded that the difficulty of modal sense classification could be eased with models that build on semantic features.

Taghipour et al. (2015) explored the option of using word embeddings in word sense disambiguation (WSD) tasks. They reported that treating a word as a discrete entity could be a weakness of supervised WSD systems. For this reason, a continuous-space representation of words which is termed as word embedding, is capable of containing valuable information. Their work investigated word embedding methods and adaptability for a word sense disambiguation setting. They also used a feed-forward neural network to adapt the word embeddings by including some selective information rather than using basic word vectors. They included information like surrounding words, collocations, and POS tags to the word embeddings. This resulted in classifying the word senses more accurately, which led to their claim that word embeddings with different information improve the accuracy of WSD systems.

The most relevant work related to Modal Sense Classification was done by Marasović et al. (2016). They explored Modal Sense Classification for both English and German languages. The modal verbs for the English language were *can*, *could*, *may*, *must*, and *should*. For the German language, the modal verbs were *dürfen* (may), *können* (can), *müssen* (must), and *sollen* (should). They worked with all three types of modalities (Epistemic, Deontic, and Dynamic). Convolutional Neural Network (CNN) was used as a classifier instead of standard Neural Networks to perform their experiments. Their experiments were based on three different datasets. To vectorize the text, they explored the impact of three different word embedding techniques called Word2Vec by Mikolov et al. (2013), dependency-based embeddings by Levy et al. (2014), and randomly initialized embeddings. They used both static and tuned versions of word embedding techniques. There were three different CNN models, out of which two were for the English language (balanced and imbalanced dataset), and the other one was for the German language. They performed the evaluation of the models using Cross Validation. For each model, there were six different word embedding techniques (Word2Vec-static & tuned, dependency-static & tuned, random-static & tuned). The accuracies of the model were quite diverse for each word embeddings with respect to the modal verbs. Regarding the English language, the modal verbs *may*, *must*, and *should* showed almost the same accuracies over all six different word embedding techniques for both of the CNNs. For the modal verb *can* and *could*, the overall accuracies were comparatively less, but there was a notable difference in terms of the word embedding techniques. For the German language, it was also found that different word embedding techniques had different effects on the CNN model. Finally, they claimed that their CNN architecture outperformed other baselines and models.

One of the most recent works related to Modal Sense Classification was done by Li et al. (2019). They also used different embedding techniques depending on the task-specific context with a simple Support vector machine (SVM) model. Their datasets were similar to Marasović et al. (2016), except they only worked with English modal verbs. They attempted shallow feature sets (POS tags, unigrams, bigrams, and trigrams) and pre-trained embeddings for the context words. They used Word2Vec with their own weighting functions. They obtained the context representation by taking the weighted sum of vectors of all context words. There were four different weighting functions: Average Weighting, Linear Weighting,

Exponential Weighting, and Modal-Specific Weighting. Their results showed that assigning more weights to the modal-specific context word can predict the modal sense more accurately.

3 Solution Architecture

To perform the Modal Sense Classification, it was necessary to divide the whole experiment into several sub-tasks. The workflow created to accomplish the sub-tasks was a one-by-one process. One task had to be done before going for another one. So, each sub-task was dependent on the other.

3.1 Environment Setup

To accomplish the entire experiment, Python 3.8.15 was used. It was necessary to use multiple packages and frameworks related to data science and machine learning to handle different tasks smoothly. Pandas 1.5.2 (The pandas development team 2022) was needed to manipulate and prepare the dataset. For handling various numerical operations, NumPy (Harris et al., 2020) 1.24.0 was used. Scikit-Learn (Pedregosa et al., 2011) 1.2.0 was used to split the dataset into the train, validation, and test set, convert the response variable to the One-hot-encoding label, and also for Cross Validation. These will be discussed in the later part of this paper. For different word embeddings, Flair (Akbi et al., 2019) 0.11.3 was used. TensorFlow (Abadi et al., 2015) 2.10 was used to create, train, and evaluate the LSTM model. It was also used to create padding in the matrix of the word embedding.

Google Colab connected with a local runtime was used for the coding environment. The local runtime was built on Miniconda 4.12.0. The technical specification of the machine used for the experiment was MacOS Ventura 13.2.1 with Apple M1 Pro chip (8-core CPU and 14-core GPU), 16 GB RAM, and 512 GB SSD. The size of the dataset was 545 KB.

3.2 Dataset

The experiment was conducted on one dataset. The dataset is based on a corpus called MASC. MASC is a Manually Annotated Sub-Corpus of the Open American National Corpus (Ide et al., 2008). This data was not annotated with the modalities. So, an annotated dataset was needed for the task. For this reason, I looked into Marasović et al. (2016), who added the modality annotations.

The dataset is comprised of sentences from the English language that contain five different English modal verbs. The verbs are *can*, *could*, *may*, *must*, *shall*, and *should*. The sentences are collected from nineteen different genres. The genres include blogs, emails, journals, newspapers, letters, essays, debates, fiction, etc. The individual sentence contains its own modality. There are three types of modalities present in the whole dataset. The modalities are Epistemic (ep), Deontic (de), and Dynamic (dy). Table 1 below shows a partial look at the dataset.

Text	Modality
What can best drive our economic engine?	dy
“ She should be here soon. ”	ep

Mileage Plus account number must be provided at time of booking.	de
I fear that may be true for me	ep

Table 1: Partial overview of MASC Dataset (Annotated)

3.3 Data Preparation

The data was contained in the text files (.txt) as plain text. The encoding format of the texts was UTF-8. The text and modality were separated by a tab delimiter. There were 114 text files in total, as there were nineteen different genres and six different modal verbs (So, $19 \times 6 = 114$). Almost all of the six modal verbs were contained in every genre except “shall”. Only a few genres, such as blogs, court transcripts, fiction, government documents, jokes, journals, movie scripts, and twitter, contained text with the modal verb “shall” out of all the nineteen genres. So, it can be deduced that shall is not regular in most of the texts. There is another exception for the genre “telephone conversation”. This is the only genre that did not have any text with the modal verb “must”.

Data cleaning was necessary to make the data suitable for the experiments, as there were some garbage values and noises in the texts and modality categories. There were 1847 texts (rows) available in the dataset after cleaning the data and removing the duplicate entries.

For the experiment, it was chosen to concentrate only on two types of modalities: Epistemic and Deontic, ignoring the Dynamic modality. The reason to exclude dynamic modality is its number of modal verbs distribution. From table 2, it can be seen that dynamic modality is mainly found in text containing the modal verbs “can” and “could” out of six different modal verbs. It can be also said that a very large number of dynamic modalities in text mainly consist of only “can” and “could” modal verbs, which are 671 and 235, to be specific. The occurrences are relatively low compared to the other two categories, such as epistemic or deontic. For epistemic and deontic, it’s only 103 for both types. It might be possible that “can” verbs are more biased toward dynamic modality. So, it is safe to concentrate only on Epistemic and Deontic modalities to avoid the imbalance in the dataset or any biases. Another reason is that many scholars suggest modalities can be divided into two major categories: Epistemic and Deontic (Zhang, 2019).

	Can	Could	May	Must	Shall	Should
Epistemic	103	129	194	27	0	25
Deontic	103	16	44	98	1	194
Dynamic	671	235	4	0	1	2

Table 2: Composition of Modalities according to modal verbs in the dataset

So, after keeping only Epistemic and Deontic modalities, there were 934 texts (rows) in total, out of which 478 texts were epistemic and 456 texts were deontic. Also, only two columns (text and modality) of the dataset will be used in the experiment.

3.4 Word Embeddings

For the LSTM model, the whole text data was used as the predictor. On the other hand, the modality was used as the response variable for the training. LSTM does not understand the string data. So, both of the columns (text and modality) needed to be converted to some numerical values. The column “text” was vectorized using the pre-trained word embedding techniques. The other column, “modality” was converted to “one-hot-encoding” labels. To feed the data into the LSTM model, the framework called “Flair” was used for vectorizing the texts.

It was decided to use two different Word Embedding techniques for the experiment. One is XLNet (Yang et al., 2019), and the other one is RoBERTa (Liu et al., 2019).

3.4.1 XLNet

XLNet is a Bidirectional Transformer with Permutation Based Modeling. In this embedding, the tokens are predicted randomly instead of sequentially. Because of the permutation technique, it should handle the word dependencies well. XLNet was trained on a dataset with a size of over 130 GB. To understand XLNet, consider the following two sentences:

4. He loves watching football.
5. He loves eating pizza.

Here, the lists of tokens for the example sentences (4) and (5) are [He, loves, watching, football] and [He, loves, eating, pizza], respectively. Notice that both of the sentences have the same structure and the same length of the list of tokens where the first two tokens are identical, and the last two tokens are different. To predict or vectorize the third token, “watching” for the example sentence (4), let’s consider a function $\text{Predict}(\text{watching} \mid \text{He, loves, football})$ is used. So, it handles all the combinations of the tokens to predict the vectors. Now for the second example sentence (5), to predict or vectorize the third token, “eating”, the same function with different parameters $\text{Predict}(\text{eating} \mid \text{He, loves, pizza})$ is used. It is possible that for the simple word embedding techniques, it would have been predicted or used the same vector for the token “watching” for the example sentence (5) also because it would only consider the previous token, which is the same (“loves”) for both of the example sentences in this scenario. So, using XLNet should allow the embeddings to be more distinct in terms of the combinations of the tokens in different texts. For this experiment, XLNet-Large-Cased was used.

3.4.2 RoBERTa

RoBERTa is a Robustly Optimized BERT pre-trained Approach with Dynamic Masking. So, different tokens will be masked for different epochs for the prediction. It is an updated version of another word embedding technique, BERT, by Devlin et al. (2019). RoBERTa was trained on a dataset with a size of over 160 GB compared to its predecessor BERT, which was trained on a dataset of 16 GB. Here is an example sentence to understand how RoBERTa works:

6. I love to travel in Europe.

To vectorize this example sentence (6), different tokens will be masked in each epoch during the training phase of this technique. For example, the token “love” will be masked in the epoch: “n” (“n” can be any natural number – 1, 2, 3, ...). So, it can be thought like this: “I ... to travel in Europe”. Now for the epoch: $n + 1$, a different token, “travel” will be masked for the same sentence. For this time, it will be something like this: “I love to ... in Europe”. For this experiment, RoBERTa-Large was used.

3.5 Model Building

3.5.1 LSTM Model

For the classification task, LSTM is used. The complete form of LSTM is Long Short-Term Memory. LSTM is a particular type of RNN where RNN means Recurrent Neural Network. In a traditional feed-forward neural network, a fixed amount of data is taken as the whole input. For this reason, a fixed amount of data is also produced as a whole output. But a recurrent neural network does not take all the input data at a time. It takes data one at a time and also takes in a sequence. At every step, the RNN does a calculation before producing an output. This output is also known as the hidden state, and later this output is combined with the next input in the sequence to produce another output. It goes like this until it reaches the final input and calculates the final output. So, the process continues till the model is finished or the input sequence is ended.

So, LSTM was chosen to be used for Modal Sense Classification because it fits perfectly for the intention and approach of the experiment. LSTM is commonly used to solve problems involving sequential data prediction. Here, the task of the model would be to predict the class based on the text data. Text data is sequential data. Like any other neural network, LSTM includes layers. These layers aid in learning and recognizing patterns in data for improved performance. The basic operation of an LSTM can be seen as holding the required data and discarding the data that is not needed or beneficial for the future prediction. It also enables the model to perform better.

Another reason to choose LSTM was to see if using LSTM can produce any better results than previous works because, to my knowledge, none of the work related to Modal Sense Classification used LSTM. Instead, all the previous works related to Modal Sense Classification were done using another popular model, CNNs (Convolutional Neural Networks).

LSTM also features a unique capability that allows it to memorize the data sequence. It could be very important for the classification experiment, because the two sentences might contain the same tokens, but the sequences of the token are different. For example, the sentence “Mr. Brown can do the job.” can be formed like this: “Can Mr. Brown do the job?” also. These two sentences have the same tokens with the same modal verb, “can” but with a different sequence. Because of the difference in the sequences, the meaning or the context can be different. The modality of the first sentence, “Mr. Brown can do the job.” can be epistemic. The reason can be that it is highly possible that Mr. Brown can perform the job. On the other hand, the example “Can Mr. Brown do the job?” is deontic because it can be a request to

someone else if Mr. Brown can perform the job. So, the sequence and the context of the data are essential for this type of experiment, and LSTM might be able to ensure the requirements.

Focusing on removing unnecessary data is an essential aspect of Machine Learning Models. It can save calculating the training time and expenses of the resources. Text data can be full of garbage and noises. Some text can contain unnecessary white spaces, punctuation, formatting, etc. But the pattern of these unnecessary things might be dynamic, which means it can depend on one's writing habits or might come irregularly. For example, some might write like "Mr. Brown always come ... office early." Here the unnecessary thing is the three dots "..." because it does not really contain any meaning or advantage to the model. LSTM can reduce the training burden of the model by omitting this type of information which saves a great deal of memory consumption and calculation time, as LSTM already tends to require more memory than some other traditional neural networks. So, LSTM is a strong tool for text categorization and other tasks that are based on text since it memorizes the sequence of information and eliminates unused information.

LSTM works in a particular way. It takes three different pieces of information, which are Current Input Data, Short-Term Memory, and Long-Term memory. From a practical point of view, it can be said that Short-Term Memory is for the short run, where the information will be kept for a short period of time. Long-Term Memory is for the long run, where the information will be kept for a long period of time combined with other information. Short-Term Memory is also known as the Hidden State. It carries information from the previous cell. Long-Term Memory is called Cell State, which has the capability to store and load information. For the Long-Term Memory (Cell State), it is not necessary that the information has to be from the immediate previous cell. It can also contain the information of other cells. The difference between RNN and LSTM is that the Short-Term Memory (Hidden State) is present in both RNN and LSTM, whereas the Long-Term Memory (Cell State) is only present in LSTM. Another important concept in LSTM is the Cell Gates. The Cell Gates control the passing of information from one cell to another cell. Before passing on the information of Long-Term Memory (Cell State) and Short-Term (Hidden State) Memory to another cell, the cell uses Gates to regulate the information. So, they decide which information is important and will be kept for further operations. Another feature is that they can also eliminate the information which is not relevant or not required for future operations. These happen in every time step. There are 3 types of gates present in Cell Gates: Input Gate, Forget Gate, and Output Gate. The Input Gate determines which data should be saved in the Long-Term Memory (Cell State). The Forget Gate determines whether Long-Term Memory (Cell State) information should be stored or eliminated. Finally, the Output Gate combines the current input with the information contained in the prior Short-Term Memory (Hidden State) and the freshly computed Long-Term Memory (Cell State). This operation creates a new Short-Term Memory (Hidden State) that is forwarded to the cell in the next time step. The Cell Gates are also called Filters.

3.5.2 Model Configuration

To train and test the model for the experiment, the whole dataset has to be divided in a certain way. The dataset will be split into three different sets. Each set is unique from one another. The reason for creating the unique sets is to make the model perform the task and make the evaluation possible in a better way. This is quite common in a machine learning task that the model will be trained and validated using two sets of data, and then it will be tested with another set of data that the model has not seen before. So, for this LSTM experiment, the whole dataset was divided into Training Set, Validation Set, and Test Set. There were 934 rows in the final prepared data, where 478 texts were epistemic and 456 texts were deontic. The data was split in a way so that 70% of the whole data was considered for the training. The rest of the 30% of the entire dataset was further divided into validation data and test data, where 50% of the data (the remaining 30%) was considered for validation, and the other 50% was considered for the test. So finally, 70% of the data was chosen to be the train set, 15% of the data was chosen for the validation set, and 15% of the data was chosen as the test set. During the split, the rows for each set were chosen by shuffling the whole data instead of row by row. The advantage of this approach is that the data variety was ensured. If the data was considered row by row in a specific order, it is possible that some biases might be introduced, which might influence the model. So, it is always better to shuffle the data. After the successful splitting, there were 653 rows in the train set, 140 rows in the validation set, and 141 rows in the test set.

After creating the train set, the validation set, and the test set, it was time to use the word embedding techniques. From the dataset, the column “text” of each set needed to be vectorized with the two chosen word embedding techniques (XLNet and RoBERTa), as this column contained the text data in string format. The One-hot-encoding label was applied for the other column, “modality” as it was the response variable. It was decided to embed and label the train set, the validation set, and the test set separately because each set serves a different purpose. It was also possible to embed and label the whole dataset first and then split the embedded and labeled data into the train set, the validation set, and the test set. As the dataset was split first, the embeddings and labeling were applied later.

The next thing was to apply padding to each set for the model. The length of the texts present in the dataset was not fixed. So, different texts had different lengths in terms of the number of tokens. For example, 76, 80, and 74 were the highest text lengths found in the train, test, and validation sets. LSTM will not work if the input length (the shape of the input matrix) differs. So, it is mandatory to fix the issue by padding the data. It is obvious to pad the data by its highest length that had been found in the data. Otherwise, the data would lose some of its information. As the highest length found in the dataset was 80, so all the sets were padded by a length of 80. After applying the padding on both of the word embeddings (XLNet and RoBERTa), the shape of the matrix was (653, 80, 1024) for the train set, (140, 80, 1024) for the validation set, and (141, 80, 1024) for the test set.

The model created for this experiment consisted of five layers in total. They were the input layer, three hidden layers, and finally, the output layer. The input layer contained a dimension size of 1024, an input shape of (80, 1024), a dropout of 20%, a recurrent dropout of 20%, and activated return sequences. After flattening, the three hidden layers were dense layers with a size of 512, 128, and 32, respectively. All the three hidden layers had the activation function “relu” and a dropout of 40%. The output layer was also a dense layer with a size of one and the activation function “sigmoid”. The loss function, optimizer, and metric that were used for the model compilation were “BinaryCrossentropy”, “Adam”, and “BinaryAccuracy”, respectively. The model had 50,406,081 parameters in total, where all of the parameters were trainable. The configurations of the model were identical for both of the word embedding techniques (XLNet and RoBERTa). This was intentional and mandatory to compare the two word embedding techniques.

3.5.3 Model Training

The model was trained for 20 epochs with a batch size of 128. Shuffling was also activated for each epoch so that different orders of the data could be trained. These training arguments were also the same for both of the word embeddings (XLNet and RoBERTa).

For the XLNet, the training of the model began with a training loss of 2.1715, training accuracy (binary accuracy) of 0.4946, validation loss of 0.7060, and validation accuracy (binary accuracy) of 0.5000. After 20 epochs, the training loss, training accuracy (binary accuracy), validation loss, and validation accuracy (binary accuracy) were 0.1448, 0.9418, 1.1461, and 0.5857, respectively.

For the RoBERTa, the model started its training with a training loss of 2.7762, training accuracy (binary accuracy) of 0.5023, validation loss of 1.4436, and validation accuracy (binary accuracy) of 0.3786. The model ended its training with a training loss of 0.2093, training accuracy (binary accuracy) of 0.9280, validation loss of 0.3966, and validation accuracy (binary accuracy) of 0.9000 after 20 epochs.

3.6 Model Evaluation

After training the LSTM model, the next part was to evaluate the model for both of the word embedding techniques (XLNet and RoBERTa). The evaluation was performed using the test data, which the model has not seen yet. For the XLNet, the evaluation loss was 0.7621, and the evaluation accuracy (binary accuracy) was 0.7376. On the other hand, for the RoBERTa, the evaluation loss was 0.4722, and the evaluation accuracy (binary accuracy) was 0.8227. This is the basic practice of evaluating a model. But for the sake of the experiment and to attain a more robust evaluation of the model, another evaluation technique called “Cross Validation” was also performed on the model of both of the word embedding techniques (XLNet and RoBERTa).

3.6.1 Cross Validation

Cross Validation is a technique for analyzing and testing the performance of a Machine Learning model. It is frequently utilized in applied Machine Learning tasks which allows to perform the comparison of

different metrics for the given predictive modeling challenge. For this experiment, it was decided to use the “K-fold” Cross Validation methodology. The term “K-fold” simply refers to how many folds are preferred to divide the whole dataset into.

For Cross Validation, the training and evaluation of the model happened quite similarly to before. Here, “StratifiedKFold” from “Scikit-Learn” was used to set up the folds of Cross Validation. It was decided to go with five folds as the number of folds. There were also some other arguments that were necessary for the folds. “Shuffle” was enabled so that the data of each fold was not the same in every loop. It would randomly shuffle the data for every fold to avoid any biases. To keep the shuffled data the same, another argument called the “random state” was used. It allowed the randomness of each fold to be controlled for each class. In the prepared dataset, there were two classes: epistemic and deontic. The “random state” argument would keep the data points of these two classes (epistemic and deontic) balanced across the folds so that during the shuffle, the number of data points for a specific class was not way too larger or smaller than the other class. It also ensured that it was the same shuffled data when the same number of folds (in this case, it was five) was applied. The “random state” was set to an integer number “7”, although it can be any integer number except “1”.

3.6.2 Data Preparation for Cross Validation

For the purpose of K-fold Cross Validation, the dataset had to be prepared a bit differently than before. This time the whole dataset would not be divided into the train set, the validation set, and the test set initially like before. Instead, the entire dataset was vectorized with each word embedding technique for its own LSTM model. One-Hot-Encoding was also performed on the whole dataset's response variable (modality). Also, there was no validation set created this time. So, the whole dataset had to be divided into a number of folds where the train set and the test set would be automatically created by Cross Validation according to its number of folds.

3.6.3 Model Configuration for Cross Validation

The model configuration for Cross Validation was quite the same as before, except for one significant change. It was the use of Bidirectional LSTM instead of regular (Unidirectional) LSTM. The reason for using the Bidirectional LSTM was to see if it could produce any better results. The main difference between Bidirectional LSTM with Unidirectional LSTM is that Bidirectional LSTM runs the input in two ways (Forward and Backward). So, Bidirectional LSTM will run its input from the past to the future (Forward) and also from the future to the past (Backward). It also uses two hidden states together to perform these “Forward” and “Backward” operations. In Unidirectional LSTM, it performs only the “Forward” operation. It is possible that Bidirectional LSTM might work better when the context is very important. Also, Bidirectional LSTMs are proven to produce better results for the classification tasks related to Natural Language Processing (NLP). As Modal Sense Classification is indeed a classification task and vastly depends on the context of the text, it was decided to go with Bidirectional LSTM for the Cross Validation.

All the other model parameters were exactly the same as before. The model consisted of five layers: one input layer, three hidden layers, and one output layer. The input layer's dimension was 1024, and the input shape was (80, 1024) with 20% of dropout, recurrent dropout, and the return sequences set to "True" (enabled/activated). After flattening, each hidden layer was a dense layer and contained a size of 512, 128, and 32, respectively, with the same activation function "relu" and a dropout of a value of 40%. In the end, the output layer was also a dense layer with the size of one and the activation function "sigmoid". The training parameters were the same as well. The number of epochs was 20, the training batch size was 128. The compiling parameters of the model were also similar to before. The loss function was "BinaryCrossentropy", the optimizer was "Adam", and the metric was BinaryAccuracy. However, just like the previous approach, it was essential to keep the configurations of the Bidirectional LSTM model for Cross Validation the same for both of the word embedding techniques (XLNet and RoBERTa) so that both could be compared. The total parameters of the model were 100,741,825. All of the parameters were trainable. It was almost the double compared to the parameter count for the model of the previous approach. The reason is the Bidirectional LSTM. Both of the word embedding techniques (XLNet and RoBERTa) had the same number of total parameters.

3.6.4 Model Training and Evaluation for Cross Validation

The training of the model for Cross Validation was performed a bit differently compared to the previous approach. The apparent reason was the K-folded data. As the dataset was divided into five folds, different combinations of the folds had to be trained with the model. The number of folds was five, so the model was trained and evaluated five times with each combination of the folds. For example:

- Out of the 5 folds, fold no. 1, 2, 3, and 4 would be used as the train set, whereas fold no. 5 would be used as the test set for the first training and evaluation, respectively.
- For the second training and evaluation, fold no. 1, 2, 3, and 5 would be the train set, and fold no. 4 would be considered as the test set.
- For the third training and evaluation, the train set would be the fold no. 1, 2, 4, and 5; the test set would be the fold no. 3.
- Similarly, in terms of the fourth training, the fold no. 1, 3, 4, and 5 were the train set. For the evaluation, it was the fold no. 2 used as the test set.
- Finally, for the fifth training, the train set was consisted of the fold no. 2, 3, 4, 5. The fold no. 1 was the test set for the evaluation.

The training methodology of the model was the same for both of the word embedding techniques (XLNet and RoBERTa). Each training and evaluation of the model was performed in a loop. As there were five folds, so there were five iterations.

For XLNet,

- Iteration no. 1 of the model training was started with a training loss of 3.0167 and a training accuracy (binary accuracy) of 0.5087. After 20 epochs, it ended with a training loss of 0.2915 and a training accuracy (binary accuracy) of 0.8889. The model accuracy of this iteration was about 81.82%.
- The second iteration of the model training was initiated with a 3.5568 training loss and 0.5288 training accuracy (binary accuracy). After 20 epochs, the model training finished with a training loss of 0.1151 and a training accuracy (binary accuracy) of 0.9639. For this iteration, the model had an accuracy of 73.80%.
- The third iteration began with a training loss of 2.4357 and a training accuracy (binary accuracy) of 0.5074. It ended with a training loss of 0.0330 and a training accuracy (binary accuracy) of 0.9906 after 20 epochs. Like the previous iteration, this iteration also had a model accuracy of 73.80%.
- The fourth iteration had a training loss of 1.9878 and a training accuracy (binary accuracy) of 0.53 for the first epoch. After 20 epochs, it had a training loss of 0.3122 and a training accuracy (binary accuracy) of 0.8768. This iteration had a model accuracy of 71.66%.
- The fifth iteration of model training commenced its first epoch with a training loss of 2.5725 and a training accuracy (binary accuracy) of 0.4933. After 20 epochs, the training loss was 0.1338, and the training accuracy (binary accuracy) was 0.9532. The model accuracy of this iteration was 70.43%.

For RoBERTa,

- The first iteration was initiated with a training loss of 3.9324 and a training accuracy (binary accuracy) of 0.5074. The training of the model was finished after 20 epochs with a training loss of 0.0542 and a training accuracy (binary accuracy) of 0.9853. The overall accuracy of the model was 82.89%.
- The second iteration started its training with a training loss of 3.3021 and a training accuracy (binary accuracy) of 0.4967. After 20 epochs, the training loss and the training accuracy (binary accuracy) were 0.062 and 0.9786, respectively. The model accuracy was 88.77%.
- The third iteration of the model training was begun with a training loss of 2.4105 and a training accuracy (binary accuracy) of 0.5207. The training of the model was ended with a training loss of 0.1523 and a training accuracy (binary accuracy) of 0.9612. The model had an accuracy of 86.63% for this iteration.
- The fourth iteration started its first epoch with a training loss of 2.8711 and a training accuracy (binary accuracy) of 0.4940. For epoch 20 at the end of the training, the training loss was 0.1280, and the training accuracy (binary accuracy) was 0.9585. This iteration had a model accuracy of 83.42%.

- Finally, the fifth iteration had a training loss of 2.4084 and a training accuracy (binary accuracy) of 0.5013 for the first epoch. The iteration ended its training with a training loss of 0.0500 and a training accuracy (binary accuracy) of 0.9826. The accuracy of this iteration was 84.41%.

Here, for both of the word embedding techniques (XLNet and RoBERTa), the model training did not have the validation loss and the validation accuracy as there was no validation set. Also, showing the evaluation loss of the models was skipped. The model accuracy was the more important aspect to look into, and it was shown in percentage.

4 Results

There were two different evaluation approaches that were used to evaluate the LSTM model to compare both of the word embedding techniques (XLNet and RoBERTa). The first evaluation approach was the basic model evaluation, and the other approach was Cross Validation.

For the basic evaluation, the LSTM model with RoBERTa, word embedding technique showed better model accuracy than the model with XLNet. The model accuracy for RoBERTa was 82.27%, with an evaluation loss of 0.4722. On the other hand, XLNet had a model accuracy of 73.76% with an evaluation loss of 0.7621. Regarding the evaluation loss, RoBERTa showed a better result than XLNet, as the evaluation loss was higher for XLNet. So, it can be said that a specific word embedding technique did have an effect in classifying the modal senses, and it was RoBERTa in this case.

To attain a more robust result for coming to a decision, the most effective evaluation technique was Cross Validation. As five-fold Cross Validation was used, there were five different accuracies for each word embedding technique. These accuracies were based on different folds of the train set and the test set. For XLNet, the accuracies were 81.82%, 73.80%, 73.80%, 71.66%, and 70.43%. The mean accuracy of this word embedding technique (XLNet) was 74.30%. The standard deviation for this technique was 3.97. On the contrary, for RoBERTa, the accuracies of the model were 82.89%, 88.77%, 86.63%, 83.42%, and 84.41%. The mean accuracy and the standard deviation were 85.22% and 2.19. Both mean accuracy and standard deviation had a better result for RoBERTa compared to XLNet. Because of the lower standard deviation, it also showed that the accuracies of RoBERTa were more closely clustered around the mean, which is obviously more reliable. So, Cross Validation just reconfirmed the result of the basic evaluation. Hence, it is strongly possible that using a different word embedding techniques indeed affected the model. That is why RoBERTa produced a better result XLNet. Table 3 shows an overall look at the results.

Word Embedding Technique	LSTM Accuracy (Basic)	LSTM Accuracy (Cross Validation)
XLNet	73.76%	74.30%
RoBERTa	82.27%	85.22%

Table 3: Model Accuracies of LSTMs with XLNet and RoBERTa

5 Conclusion and Future Scope

In this experiment, two different word embedding techniques (XLNet and RoBERTa) were used with LSTM to perform Modal sense Classification. It was already known that Modal Sense Classification is heavily influenced by the context and the meaning in the modal's scope. Using two most recent large pre-trained word embedding techniques enabled the LSTM model to learn semantic features of the text more efficiently and automatically without intensive feature engineering. The two word embedding techniques used here were working differently in terms of their vectorizing methodology on words. Each of the word embeddings tried to self-learn the context or representation of words in a text on their own. Also, using LSTM is quite noble when it comes to text classification. The LSTM showed that the word embedding technique: RoBERTa was better than XLNet. Moreover, a Bidirectional LSTM architecture was set up for Cross Validation to obtain a more robust result of the model, which reconfirmed the previous findings. It is possible that RoBERTa can unearth the contextual meaning regarding the modal's scope of a given text in a better way where XLNet is a bit less efficient.

It is possible to use other word embedding techniques with the same LSTM architecture explained here, which may allow another word embedding technique to have more accurate results than RoBERTa. The whole experiment can be easily adapted with more data and even with other languages.

6 References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G.s., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Kaiser, L., Kudlur, M., Levenberg, J., & Zheng, X. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. *CoRR*, *abs/1603.04467*.
- Akbik, A., Blythe, D., & Vollgraf, R. (2018). Contextual String Embeddings for Sequence Labeling. *Proceedings of the 27th International Conference on Computational Linguistics*, (pp. 1638–1649). Association for Computational Linguistics.
- Devlin, J., Chang, M., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Minneapolis, Minnesota, (pp. 4171–4186). Association for Computational Linguistics.
- Harris, C., Millman, K., Walt, S., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M., Brett, M., Haldane, A., Río, J., Wiebe, M., Peterson, P., & Oliphant, T. (2020). Array programming with NumPy. *Nature*, *Volume 585*, (pp. 357-362). 10.1038/s41586-020-2649-2.

- Ide, N., Baker, C., Fellbaum, C., & Fillmore, C. (2008). MASC: The Manually Annotated Sub-Corpus of American English. *Proceedings of the Sixth International Conference on Language Resources and Evaluation (LREC-2008)*.
- Levy, O., & Goldberg, Y. (2014). Dependency-Based Word Embeddings. *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics, Volume 2: Short Papers, Baltimore, Maryland*, (pp. 302–308).
- Li, B., Dehouck, M., Denis, P. (2019). Modal Sense Classification with Task-Specific Context Embeddings. *ESANN 2019 - 27th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning, Bruges, Belgium*. fhal-02143762f.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. *ArXiv*, abs/1907.11692.
- Marasović, A., & Frank, A. (2016). Multilingual Modal Sense Classification using a Convolutional Neural Network. *Proceedings of the 1st Workshop on Representation Learning for NLP*, (pp. 111–120). Association for Computational Linguistics.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., & Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. *Advances in neural information processing systems 26, NIPS (2013)*, (pp. 3111–3119).
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, Volume 12*, (pp. 2825–2830).
- Ruppenhofer, J., & Rehbein, I. (2012). Yes we can!? Annotating English Modal Verbs. *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12), Istanbul, Turkey*, (pp. 1538–1545). European Language Resources Association (ELRA).
- Taghipour, K., & Ng, H. T. (2015). Semi-Supervised Word Sense Disambiguation using Word Embeddings in General and Specific Domains. *The 2015 Annual Conference of the North American Chapter of the Association for Computational Linguistics*, (pp. 314–323).
- Yang, Z., Dai, Z., Yang, Y., Carbonell, J., Salakhutdinov, R., & V. Le., Q. (2019). XLNet: Generalized Autoregressive Pretraining for Language Understanding. *Proceedings of the 33rd International Conference on Neural Information Processing Systems, Article 517*, (pp. 5753–5763).
- Zhang, J. (2019). A Semantic Approach to the English Modality. *Journal of Language Teaching and Research, Volume 10*, (pp. 879). 10.17507/jltr.1004.28.

Zhou, M., Frank, A., Friedrich, A., & Palmer, A. (2015). Semantically Enriched Models for Modal Sense Classification. *Proceedings of the First Workshop on Linking Computational Models of Lexical, Sentential and Discourse-level Semantics, Lisbon, Portugal*, (pp. 44–53). Association for Computational Linguistics.

Please always attach a **signed declaration of independence** of your work:

I, Mohammad Shafiqul Islam, confirm that the work for the term paper with the title: "Dependency of Modal Sense Classification on Different Word Embedding Techniques using LSTM" was solely undertaken by myself and that no help was provided from other sources as those allowed. All sections of the paper that use quotes or describe an argument or concept developed by another author have been referenced, including all secondary literature used, to show that this material has been adopted to support my work.

Place / Date

 20.04.2023

Signature

